GAME ARCHITECTURE BLUEPRINT

Location-Based Interactive Streaming & Social Gaming Platform

DOCUMENT TYPE:	PHASE:	FRAMEWORK:	TARGET PLATFORMS:
Technical Spec	Phase 2 (Architecture)	Expo Router & React Native	iOS / Android Mobile

1. EXECUTIVE PRODUCT VISION

The platform is a **next-generation social, location-aware multiplayer game** combining physical world exploration with real-time interactive media streaming. Unlike static single-player location games, this product merges live video/audio interaction, real-time game state mirroring, and modular seasonal events into a single battery-optimized mobile architecture.

CORE **Location Exploration**

Real-world map navigation built on an ultra-lightweight 2D/3D canvas engine. Deliberately excludes heavy camera Augmented Reality (AR) to guarantee 60 FPS performance and maximum battery life.

MEDIA **Interactive WebRTC Streaming**

Supports 1-way host broadcasts, multi-way live audio/video rooms, and WebRTC DataChannel state sync for zero-overhead gameplay mirroring across viewers.

BATTERY **OLED Theme Optimization**

Static Design Token theme system featuring true-black OLED dark modes (#000000) that physically shut off pixels to offset GPS display drain by up to 30-60%.

EXTEND **Modular Seasonal Content**

Architected via the Open/Closed Principle (OCP) to allow instant injection of seasonal themes, Halloween/Winter map shaders, and custom emotes without altering core engine loops.

2. 3-LAYER HYBRID SYSTEM ARCHITECTURE

To prevent video rendering and GPS tracking from blocking the main JavaScript UI thread, the application is divided into three distinct execution layers:

LAYER 1: REACT NATIVE APP & UI SHELL (Cold Path)

Principles: Convention over Configuration (CoC), SOLID (SRP, OCP), Design Tokens.
Responsibilities: Expo Router file navigation (`app/_layout.tsx`), floating video call overlays, theme context switching, emote drawers, inventory management, and orientation adapters (Portrait/Landscape).

LAYER 2: REAL-TIME MEDIA & STREAMING ENGINE (WebRTC + SFU)

Principles: Selective Forwarding Unit (SFU) Architecture (e.g., LiveKit / Agora), State Syncing.
Responsibilities: Low-latency (<200ms) multi-way video/audio routing, WebRTC DataChannels for ultra-lightweight game state binary broadcasts (gameplay mirroring without video encoding).

LAYER 3: HOT-PATH MAP & GAME ENGINE (Native Canvas)

Principles: Data-Oriented Design, Entity Component System (ECS), Native Canvas rendering.
Responsibilities: Background GPS geolocation tracking, sprite rendering, physics calculations, and particle effects running at 60 FPS off the main JS thread.

3. CORE ARCHITECTURAL PRINCIPLES APPLIED

PRINCIPLE	IMPLEMENTATION STRATEGY	PRIMARY TECHNICAL BENEFIT
Convention over Configuration	Expo Router file-system routing (`app/` folder structure, `_layout.tsx`, `+not-found.tsx`).	Zero boilerplate router configs; predictable deep linking and web/mobile compatibility.
Open/Closed Principle (OCP)	Pluggable `WinAnimation`, `EventStrategy`, and `EmoteConfig` interfaces.	New seasonal events and emotes are added without editing or risking core game loop code.
Single Responsibility (SRP)	Strict separation between UI components, Custom Hooks (`useLayout`, `useTheme`), and API services.	Screens remain lightweight UI shells; business logic can be tested independently.
State Mirroring (over Video)	Broadcasting coordinate payloads over WebRTC DataChannels rather than screen-recording pixels.	Reduces bandwidth by 99%, prevents phone overheating, and maintains native 60 FPS rendering.

4. RECOMMENDED PHASE 2 DIRECTORY STRUCTURE

Uses a **Feature-First (Domain-Driven) Folder Architecture** to cleanly separate business modules from Expo Router navigation layout routes:

```
my-game-app/
├── app/                          <-- ROUTING LAYER (Convention over Configuration)
│   ├── _layout.tsx               <-- Root Layout & Global Context Providers
│   ├── +not-found.tsx            <-- Fallback 404 Route
│   ├── index.tsx                 <-- World Map Main Screen
│   ├── (auth)/                   <-- Authentication Stack
│   └── (modals)/                 <-- Overlay Sheets (Video Call, Settings, Emote Picker)
│       ├── live-stream.tsx
│       └── profile.tsx
├── features/                     <-- BUSINESS DOMAIN LAYER (SOLID / OCP)
│   ├── map/                      <-- Map Engine Components & GPS Hooks
│   ├── streaming/                <-- WebRTC SFU Services & State Sync
│   └── seasonal/                 <-- Pluggable Holiday Events & Shaders
├── theme/                        <-- DESIGN TOKEN SYSTEM (OLED Dark Mode)
│   ├── tokens/ (colors, spacing)
│   └── ThemeProvider.tsx
└── components/                   <-- REUSABLE ATOMIC UI (LSP / SRP)
    └── ScreenContainer.tsx
```

5. RESOURCE & BATTERY OPTIMIZATION STRATEGY

CRITICAL ARCHITECTURAL DECISION: NO HEAVY AR CAMERA ENGINE

Standard AR requires simultaneous CPU/GPU camera feeds, 3D tracking, and surface detection, which drains mobile battery within 30 minutes. By utilizing a high-performance 2D/3D stylized shader canvas instead, battery life is extended 3x while preserving smooth 60 FPS gameplay on all devices.

- **Hot Path vs. Cold Path Isolation:** Cold UI updates (theme state, drawer slides) run purely in React Native. Hot-path gameplay math runs inside native canvas loops without crossing the JS bridge per frame.
- **OLED True-Black Theme:** In dark/night mode, backgrounds use pure `#000000` to completely deactivate OLED pixels during outdoors gameplay.
- **Selective Forwarding Unit (SFU):** Multi-person video streams connect through an SFU server, requiring each phone to upload only a single video stream rather than mesh P2P connections.